Saber Recorder — Developer Documentation

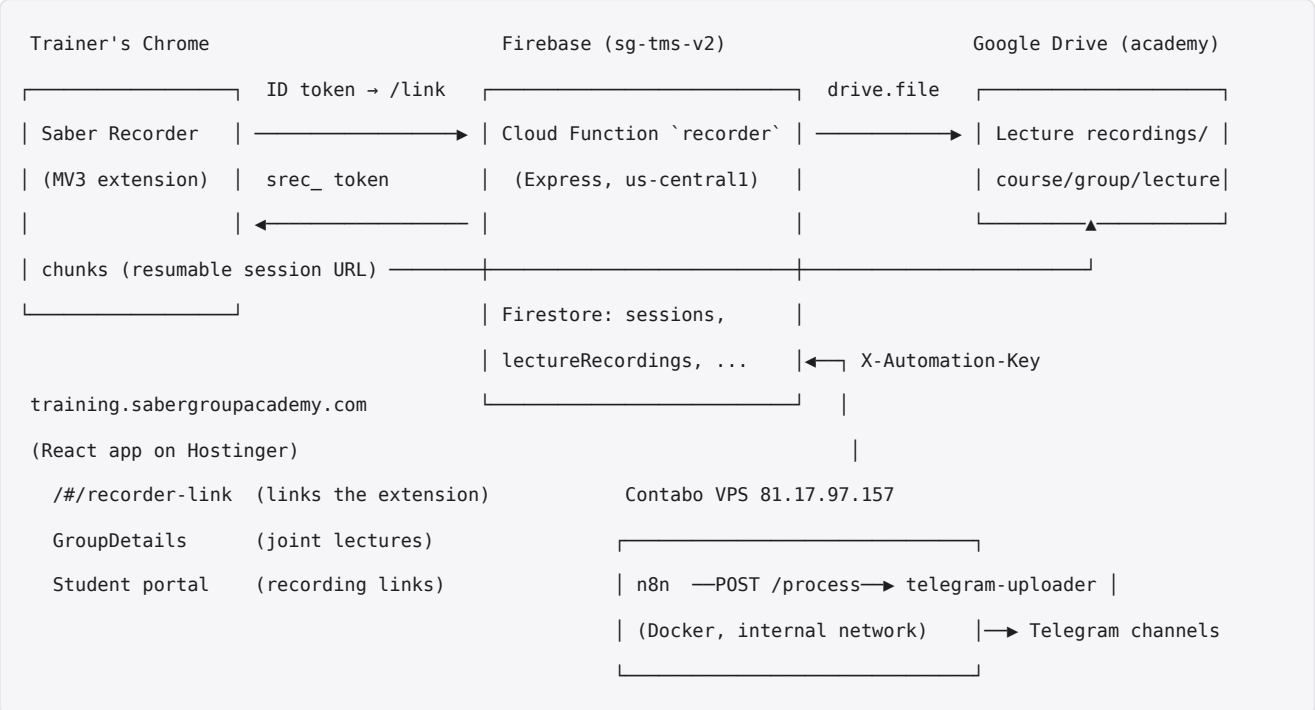
Everything a developer needs to understand, run and change the lecture recording pipeline of Saber Group Academy: the Chrome extension, the backend in Firebase, the academy Google Drive, the Telegram uploader on the VPS, the n8n automations, and the related system features (joint lectures, group archiving).

> No secret values are written in this file. Where a secret is needed, this document says **where it lives**, never what it is.

---

1. What the system does (one paragraph)

A trainer records an online lecture with the **Saber Recorder** Chrome extension. The recording is saved on the trainer's computer and, at the same time, uploaded in chunks to the **academy Google Drive** (`sabergroup.eg@gmail.com`) under `تسجيلات المحاضرات / <course> / <group> / محاضرة NN - <trainer>`. When the upload finishes, the backend marks it as waiting for Telegram. Every 10 minutes **n8n** asks the **telegram-uploader** service on the Contabo VPS to download new recordings from Drive, convert them to MP4, post them to each group's Telegram records channel with the academy Telegram account, and send the message link back to the backend, which writes it into the lecture (`sessions.lectureRecordingUrl`) so students see it in the portal. Once a month n8n archives the Drive folders of groups that finished more than 6 months ago.



---

2. Resources inventory

Resource	What / where	Notes
GitHub — system repo	`SaberGD/training-sys-updated-august-2026`, branch `claude/recorder-academy-drive`	All backend, system UI, `telegram-uploader/`, `extension/meet-recorder/` (copy of the extension). **Not merged to `main` yet** (see §12).
GitHub — extension repo	`SaberGD/saber-recorder` (private), branch `main`	Standalone extension source + `scripts/package.sh`. Same code as `extension/meet-recorder/`.
Firebase project	`sg-tms-v2`	Functions `recorder` (HTTPS, `us-central1`) and `syncJointLecture` (Firestore trigger, `europe-west1`, same region as the database).
Firebase secrets	Secret Manager in `sg-tms-v2`	`RECORDER_AUTOMATION_KEY` (shared with telegram-uploader and n8n), plus the existing `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `ENCRYPTION_KEY` used to reach the academy Google account.
Academy Google account	`sabergroup.eg@gmail.com` (5 TB Drive)	Connected in the system under **Google Integrations** as the central account, with scope `drive.file`.
Google Cloud OAuth client	Same client the system already uses	Picker API key is injected at build time only (`VITE_GOOGLE_PICKER_API_KEY`), never committed.
Website	Hostinger, account `u570689065`, domain `training.sabergroupacademy.com`	Static Vite build (HashRouter). Uploaded with the Hostinger file API (TUS).
Extension download	`https://training.sabergroupacademy.com/downloads/Saber-Recorder.zip` (latest) and `/downloads/Saber-Recorder-<version>.zip`	Plain files in `public_html/downloads/`.
VPS	Contabo Cloud VPS, `81.17.97.157`	Also runs Chatwoot, Traefik and n8n (**do not restart or modify them**). SSH: `ssh -i ~/.ssh/contabo_claude -o IdentitiesOnly=yes root@81.17.97.157` (key is on the admin's Mac only).

Resource	What / where	Notes
telegram-uploader	Code `/opt/telegram-uploader`, data `/srv/telegram-uploader` (mounted at `/data`), compose project `telegram-uploader`	Joins n8n's Docker network (`chatwoot_chatwoot-net`) with alias `telegram-uploader`. No public port.
Automation key on VPS	`/root/.recorder-automation-key` (mode 600)	`install.sh` writes it into `/opt/telegram-uploader/deploy/.env`.
Telegram	Academy Telegram account + app "lectures uploader" on my.telegram.org	`api_id` / `api_hash` and the session live only in `/srv/telegram-uploader/telegram.json` and `academy.session`.
n8n	`https://n8n.engsaber.space`	Workflows in §8.
Extension ID	`koaoinnnobkcaabcgnhiajnolamnceoi`	Fixed by the `key` field in `manifest.json` (**never change it**). Allowed in `lib/recorderExtension.ts`; extra IDs via `VITE_RECORDER_EXTENSION_IDS`.
video-server/	Folder in the system repo	Self-hosted video server experiment. **Stopped / not used.**

---

3. Chrome extension (`SaberGD/saber-recorder`)

MV3, Chrome 116+. Current version **1.3.8** (Saber Group redesign by the academy, joint lecture label, fixed popup size, A−/A+ UI size, meeting recordings, MP4 recording).

File	Role
`manifest.json`	Permissions: `activeTab`, `tabCapture`, `scripting`, `storage`, `unlimitedStorage`, `downloads`, `notifications`; hosts `googleapis.com`, `cloudfunctions.net`; `externally_connectable` only for `training.sabergroupacademy.com`. Fixed `key`.
`config.js`	`SYSTEM_ORIGIN`, `LINK_PAGE_URL`, `RECORDER_API`, upload chunk size, quality presets, `DEFAULT_OPTIONS` (save local + Drive, quality `high`, auto-stop on). `OAUTH_CLIENT_ID` stays empty (the extension never gets a Google token).
`background.js`	Opens the recorder panel, REC badge, linking messages from the system page (`link-page-ready`, `recorder-linked`), finds interrupted recordings at startup.
`popup.*`	Setup: lecture picker, sources, quality, save targets, recordings that need attention, recent recordings.

File	Role
`recorder.*`	Recording panel: capture (screen + system/tab audio + mic, mixed with WebAudio), pause, per-source mute, auto-stop when the meeting ends (20 s countdown), confirm lecture after stopping, “record part 2”.
`lib/system.js`	Calls to the backend with the `srec_` token.
`lib/lecture-picker.js`	Group/lecture selection, detection from the Meet URL, joint lecture label.
`lib/uploader.js`	Drive resumable upload in chunks, resumes after network loss.
`lib/local-file.js` + `lib/storage-worker.js`	Writes the recording to OPFS every 2 s (SyncAccessHandle worker) so a closed tab, crashed browser or power cut loses only ~2 s.
`lib/webm-duration.js`	Adds the duration to the WebM header so seeking works.
`lib/recordings.js`	Recording state in `chrome.storage`.
`theme.js`	Light/dark theme and A−/A+ UI size, saved per device in `localStorage`.

Format: MP4 (H.264 + AAC, `video/mp4;codecs=avc1.640028,mp4a.40.2`, MediaRecorder timeslice 2 s, `videoKeyFrameIntervalDuration: 2000` so seeking works; without it Chrome puts minutes between keyframes in screen recordings). Current Chrome writes fragmented MP4 progressively, so a closed tab or crash keeps what was recorded (a truncated fMP4 still plays). Browsers without MP4 recording fall back to WebM (VP9 + Opus); `lib/webm-duration.js` only applies to that fallback. The VPS copies H.264 video as-is when building the Telegram MP4 (re-encodes only other codecs or > 1080p).

Packaging: `bash scripts/package.sh` → `dist/Saber-Recorder-<version>.zip`. Bump `version` in `manifest.json` and the `v1.x.x` label in `popup.html`. Keep `extension/meet-recorder/` in the system repo identical (it has no `scripts/`, `docs/`).

Publishing a new version: upload the zip to Hostinger as both `downloads/Saber-Recorder-<version>.zip` and `downloads/Saber-Recorder.zip` (see §10). If it is ever published to the Chrome Web Store, the store gives a new extension ID, which **must** be added to `VITE_RECORDER_EXTENSION_IDS` (the link page only talks to known IDs). `externally_connectable` lists web origins, so it stays as is.

---

4. Backend: Cloud Function `recorder` (`functions/recorder.js`)

Express app created by `createRecorderApp({admin, db, getDriveAccessToken, getAutomationKey, fetchImpl, now, jointLectures})`, exported in `functions/index.js` with `secrets: [GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, ENCRYPTION_KEY, RECORDER_AUTOMATION_KEY]` (per-function secrets replace the global ones, so the Google ones are listed again). The academy Drive token comes from `getCentralOAuthClient(null)`.

Base URL: `https://us-central1-sg-tms-v2.cloudfunctions.net/recorder`

Auth

- **Staff (Firebase ID token)**: `Authorization: Bearer <Firebase ID token>`. Active, non-disabled users with role in `admin, coordinator, team_leader, trainer, supervisor`.

- **Extension (`srec_` token)**: issued by `/link`, stored only as SHA-256 in `recorderTokens`; revoked with `/unlink` or when the account is disabled / role changes.
- **Automation**: header `X-Automation-Key` = `RECORDER_AUTOMATION_KEY` (constant-time compare).

Endpoints

Method & path	Auth	Purpose
`POST /link`	ID token	Issue an extension token (called by `/#/recorder-link`).
`GET /me`, `POST /unlink`	srec	Who am I / revoke.
`GET /groups`	srec	Active groups, the trainer's first.
`GET /groups/:id/sessions`	srec	Lectures of a group, with `myParts` and `jointGroups` count.
`GET /detect?meet=`	srec	Picks a lecture **only** when the open Meet link is the stored link of a lecture dated today (one match, or all copies of one joint lecture). Otherwise `{match:null, reason: no_meet / no_lecture_today / ambiguous}` and the trainer chooses; there is no guess from the schedule. The extension also preselects only a confident `meet` match.
`POST /uploads`	srec	Create/find folders (lock in `recorderDriveFolders`), number the part, open a Drive resumable session. Returns `{recordingId, uploadUrl, folderPath, fileName, part}`. The extension uploads bytes straight to `uploadUrl`. With `{kind:"meeting", title}` (no group/lecture) the file goes to `تسجيلات عامة / YYYY-MM / <title> - <trainer> - <date time>` and is never queued for Telegram.
`POST /uploads/:id/complete`	srec	Verify the file is in the recording's folder; if the trainer changed the lecture, move + rename; set `status: uploaded`, `telegramStatus: pending`. For a joint lecture, create Drive shortcuts in the other groups' lecture folders.
`GET /automation/pending`	key	Recordings waiting for Telegram (oldest first). Meetings and anything without `groupId` / `sessionId` are never listed, and `claim` refuses them (`telegramStatus: skipped`).
`POST /automation/recordings/release`	key	Called by telegram-uploader on startup: every recording left in `processing` (the previous instance stopped mid-upload) goes back to `pending` at once. A claim older than 3 h is also listed by `/pending` again.
`POST /automation/recordings/:id/claim`	key	Lock a recording (`processing`); returns `driveFileId`, a short-lived academy `accessToken`, and `targets[]` = one `{sessionId, chat, caption}` per group channel (joint lectures have several). No channel → `no_channel` (422).

Method & path	Auth	Purpose
`POST /automation/recordings/:id/result`	key	`{ok:true, links:[{sessionId, messageLink}]}` (or legacy `messageLink`) → `telegramStatus: done`, writes `lectureRecordingUrl` (part 1, or if empty) and `lectureRecordingParts[]` on each session. `{ok:false, error}` → retry, `failed` after 3 attempts.
`POST /automation/archive-groups[?dryRun=1][&months=6]`	key	Move Drive folders of finished groups to `<course>/Archive` (see §7).
`POST /joint-lectures/preview`	ID token	Numbering preview per group (no writes).
`POST /joint-lectures`	ID token	Create a joint lecture (see §6).
`POST /joint-lectures/:id/cancel`	ID token	Cancel before it starts; restores numbering.

Drive layout & scope

Meeting recordings (the extension's "تسجيل اجتماع" option) live apart, in `تسجيلات عامة / YYYY-MM /`, and have no group, lecture or Telegram step.

Lectures: `تسجيلات المحاضرات / <courseName> / <batchCode or group name> / محاضرة NN - <trainer name>` and files named `<batchCode> - محاضرة NN[ - جزء N] - YYYY-MM-DD HH.MM.mp4` (Cairo time). The scope is `drive.file`: the backend only sees folders/files it created, which is why archiving and moving must be done by the backend, not by n8n or a person's script.

---

5. Firestore collections (server-side only unless noted)

Collection	Content
`recorderTokens/{sha256}`	`uid, email, name, role, createdAt, lastUsedAt, revokedAt, userAgent`
`recorderDriveFolders/{sha256(parentId/name)}`	`parentId, name, folderId, status (creating/ready), lockedAt` — one folder per name even with concurrent uploads; dead folders are recreated.

Collection	Content
`lectureRecordings/{id}`	`kind: "meeting"` for meeting recordings (then only `title`, no lecture fields, no `telegramStatus`); otherwise lecture fields (`groupId, groupName, batchCode, courseName, sessionId, sessionNumber, sessionDate, jointLectureId`), trainer, `folderId/Path, fileName, part, mimeType, status (uploading/uploaded), driveFileId, webViewLink, bytes, durationSec, recordedAt, uploadedAt`, `telegramStatus (pending/processing/done/failed/no_channel), telegramAttempts, telegramError, telegramMessageLink, telegramLinks[], jointShortcuts[]`.
`jointLectures/{id}`	`date, time, title, meetingLink, note, status (scheduled/cancelled), groups[{groupId, sessionId}], groupIds, createdBy*, cancelled*`
`sessions` (existing, client-readable)	New fields: `lectureRecordingParts[]`, `isJoint, jointLectureId, jointGroups[{groupId, groupName, batchCode, sessionId}], jointTime`. `lectureRecordingUrl` is written by the backend.
`groups` (existing)	Read: `courseName, batchCode, telegramRecordsLink, archived, totalSessions, feedbackSessions`.
`groupExecutionPlans` (existing)	Plan items are renumbered for joint lectures.

---

6. Joint lectures (`functions/jointLectures.js`)

One lecture attended by several groups, possibly at a different position in each group's plan.

- **Create** (any staff, incl. trainers; runs server-side because trainers can't renumber other groups' sessions): in a transaction, for each group the number is `1 + max(sessionNumber of lectures that are done or dated before the joint date)`. Lectures with that number or higher move up by one (dates unchanged); `totalSessions + 1`; `feedbackSessions` and non-completed `groupExecutionPlans` items ≥ number shift +1. One session doc per group with `isJoint, isExtra, jointLectureId, jointGroups, lectureTitle, meetingLink, jointTime`.
- **Same day as a regular lecture not yet held** → the joint one takes its turn, the regular one moves after it.
- **Running it**: trigger `syncJointLecture` (onDocumentUpdated `sessions/{id}`) copies the shared execution fields (`status, start/endTimeActual, smartAssistantState, actualTrainer*, hours, breaks, practices, attendanceCode/ExpiresAt, meetingLink, lectureTitle`, …) to the other copies; echoes write nothing.
- **Attendance QR**: a joint lecture's QR from any group checks the student into **their own group's** copy (StudentPortal).
- **Payroll**: `TrainerKPIs` dedupes by `jointLectureId` (counted once).

- **Schedule regeneration** (`regenerateUpcomingSessions`) keeps joint sessions and numbers around them.
- **Recording**: stored once on Drive (+ shortcuts), uploaded once to Telegram and re-sent to the other channels without re-uploading; each lecture gets the link from its own group's channel.
- **Cancel**: only if no copy has started; reverses all of the above.
- UI: `components/JointLectureModal.tsx` , `lib/jointLectures.ts` , badge/cancel in `pages/GroupDetails.tsx` .

---

7. Group archiving on Drive

`POST /automation/archive-groups` (monthly from n8n): for every group that has a folder, if its **last scheduled lecture** is older than 6 months (Cairo date), its folder moves into `<course>/Archive` (created on demand). If an archived group gets new lectures, it moves back on the next run. `?dryRun=1` only reports. The group in the system is **not** marked archived. Response: `{cutoff, dryRun, archived[], restored[], active, errors[]}` .

---

8. n8n workflows (`https://n8n.engsaber.space`)

Workflow	ID	Schedule	Does	Status
Saber Recorder - Upload recordings to Telegram	`cfq1dRckxK0ONPiG`	every 10 min	`POST http://telegram-uploader:8080/process` (returns at once; `busy` if a run is going)	**Active**
Saber Recorder - Archive finished groups	`Z0ddyi1A9XthKfvJ`	1st of month 03:00	`POST …/recorder/automation/archive-groups` with Header Auth credential `X-Automation-Key`	Needs the credential, then activate

Manual test of the uploader: `POST http://telegram-uploader:8080/process?wait=1` (returns results), `GET /status` , `GET /health` — only reachable from inside the Docker network (e.g. an n8n HTTP node).

---

9. telegram-uploader (`telegram-uploader/`)

Python 3.12, Telethon (user account → 2 GB file limit), aiohttp, ffmpeg.

- **Loop per recording**: claim → download from Drive with the token from claim → `ffmpeg` to H.264/AAC MP4 with `+faststart` (H.264 ≤ 1080p with keyframes ≤ 5 s apart is copied as-is; anything else is re-encoded with 3 threads and a forced keyframe every 2 s, so Telegram seeking works) → thumbnail → upload to the first target channel → re-send the same media to the other targets (joint lectures) → report links. Work files in `/data/work/<id>` are deleted after each recording. FloodWait stops the batch until the next run. `BATCH_LIMIT` (default 5) per run.

- **HTTP**: `/process`, `/status` (includes `current`: recording id, step `downloading/converting/uploading/sharing`, percent), `/health` on port 8080, opened before connecting to Telegram.
- **Limits**: one recording may take at most `JOB_TIMEOUT_MIN` (150) minutes, and a Telegram upload with no progress for `UPLOAD_STALL_MIN` (10) minutes is cancelled; both report a failure, so the recording is retried on the next run (up to 3 attempts). On startup the service calls `/automation/recordings/release`.
- **Config (env)**: `RECORDER_API`, `DRIVE_API`, `AUTOMATION_KEY`, `DATA_DIR` (always `/data` inside the container), `FFMPEG_THREADS`, `BATCH_LIMIT`.
- **Install / update** (on the VPS, from a checkout of the repo): `bash telegram-uploader/deploy/install.sh` — copies code to `/opt/telegram-uploader`, writes `.env` from `/root/.recorder-automation-key`, builds and starts, waits for `/status`.
- **Telegram login (once, interactive)**: `cd /opt/telegram-uploader/deploy && docker compose -p telegram-uploader stop uploader && docker compose -p telegram-uploader run --rm uploader python login.py; docker compose -p telegram-uploader up -d`
- **Logs**: `docker compose -p telegram-uploader logs --tail 50 uploader`.
- The academy account must be **admin with "Post messages"** in every group records channel. Channel links accepted: `t.me/+invite`, `t.me/joinchat/…`, `t.me/<username>`, numeric id. Private channels give `https://t.me/c/<id>/<msg>` links (open for channel members only).

---

10. Deploying

What	How
Functions	`cd functions && npm test` (17 tests) then `firebase deploy --only functions:recorder,functions:syncJointLecture --project sg-tms-v2`
Website	`VITE_GOOGLE_CLIENT_ID=… VITE_GOOGLE_PICKER_API_KEY=… npx vite build --outDir <dir>` then upload every file to `public_html` with the Hostinger file API: `hosting_files_generate-upload-url` → for each file `POST {url}/{path}?override=true` (`Upload-Length`) then `PATCH` the bytes (`Tus-Resumable: 1.0.0`, headers `X-Auth`, `X-Auth-Rest`). Upload `assets/` first and `index.html` last. GitHub Actions (`.github/workflows/deploy.yml`) does the same on push to `main`, but the account's Actions minutes ran out.
Extension	§3
telegram-uploader	§9
Typecheck	`npm run lint` (`tsc --noEmit`). If it runs out of memory, temporarily move `dist/` and `functions/node_modules/` out of the folder.

---

11. Tests

- `functions/recorder.test.js` — `node --test` with in-memory Firestore and Drive fakes: linking, detect, folder tree under concurrency, trashed folders, complete/move, parts, Drive scope error, automation claim/result/retries, archiving, joint lectures (numbering, cancel, sync, recording fan-out).
- `telegram-uploader/tests/test_uploader.py` — `python -m unittest discover -s tests` (needs ffmpeg, aiohttp, telethon): conversion (H.264/AAC faststart), upload + link, FloodWait, failures, joint lecture fan-out.

---

12. Open items / known caveats

- **Branch not merged**: everything lives on `claude/recorder-academy-drive`; the live site and functions were deployed from it manually. Merge to `main` when CI minutes are back, otherwise the next deploy from `main` would remove these features.
- **Firestore rules for** `googleConnections` are open (`allow read, write: if true`) — tracked as a separate security task.
- `firebase-functions` package is old (deploy warning); upgrade separately.
- Recordings uploaded by the backend set `lectureRecordingUrl` with the admin SDK, so the client-side “new recording” notification in `updateSession` does not fire for them.
- `video-server/` is an unused experiment.
- Safety rules on the VPS: never restart/modify Chatwoot, n8n or Traefik, no `apt upgrade`, no `ufw` changes.